

```
#pip install nltk
```

```
import nltk
import re
import string
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize,sent_tokenize
from nltk.stem import PorterStemmer,WordNetLemmatizer
from nltk.corpus import wordnet
print("libraries imported successfully")
```

```
libraries imported successfully
```

```
from google.colab import files
uploaded = files.upload("User/sDELL/Downloads/Review.csv")
```

 Review.csv

Review.csv(text/csv) - 13152135 bytes, last modified: 5/2/2026 - 100% done
Saving Review.csv to User/sDELL/Downloads/Review.csv/Review.csv

```
data=pd.read_csv("User/sDELL/Downloads/Review.csv/Review.csv")
print(data.head())
```

	sentiment	review
0	Negative	I had no background knowledge of this movie be...
1	Negative	I am a huge Jane Austen fan and I ordered the ...
2	Negative	Nothing to say but Wow! Has anyone actually ha...
3	Negative	i like Jane Austin novels. I love Pride and Pr...
4	Negative	In this day and age of incredible special movi...

```

import re
import string

def preprocess_text(text):
    text = text.lower()
    text = re.sub(r'http\S+', '', text)
    text = re.sub(r'<.*?>', '', text)
    text = text.translate(str.maketrans('', '', string.punctuation))
    text = re.sub(r'\d+', '', text)
    text = re.sub(r'\s+', ' ', text).strip()
    return text

data['review'] = data['review'].fillna('')

cleaned_texts = [preprocess_text(text) for text in data['review']]

print("\nCleaned_texts:")
for i, text in enumerate(cleaned_texts):
    print(f"{i+1}. {text}")

```

Cleaned_texts:

1. i had no background knowledge of this movie before i bought it but it sounded cool and ive been wanting to
2. i am a huge jane austen fan and i ordered the movie from amazonuk just so i could see it without waiting for
3. nothing to say but wow has anyone actually had somebody sneak up on them in an open field well this happens
4. i like jane austin novels i love pride and prejudice and sense and sensibility books and movies and im half
5. in this day and age of incredible special movie effects this one was a sore disappointment the actors seemed
6. becky harris plays the female shopper whose misfortune it is to be in the store at the wrong time and obvious
7. watching truth about love is this a double entendre about the star is like plugging in white noise or manu
8. a painfully protracted maudlin and predictable drama my twentyfifth sidney lumet film garbo talks gets filed
9. lets be honest as a film school project made without budget and real actors this is a passably interesting
10. a terrible movie as everyone has said what made me laugh was the cameo appearance by scott mcnealy giving
11. there are plenty of comments already posted saying exactly how i felt about this film so ill keep it short
12. a film written and directed by neil young greendale is little more than an minute music video set to a do
13. i will admit that i did not give this movie much of a chance i decided pretty early on that this just wasn
14. i am as many are a fan of tony scott films when this movie came out i had high hopes that it would be like
15. this review may contain some spoilers the remake of the classic car chase movie gone in seconds begins wel
16. what an awful show science fiction fans seem to watch anything anymore regardless of quality it shocks me
17. slight spoilers but it doesnt matter anyway an exercise in gobblygook of catastrophic proportions not ever
18. i went in not knowing anything about this movie and i walked out in an half hour knowing everything about

19. i read the back of the box and it talked about mary shelley and percy shelley and lord byron i thought wor
 20. this is not michael madsens fault he was hardly in it this movie was just awful if you want to laugh and l
 21. i was fortunate enough to meet george pal and still have my dstmob poster autographed by him at a conventi
 22. if you were to judge based on the movie alone the committee that gave the stage musical a chorus line a pu
 23. i usually try to be professional and constructive when i criticize movies but my god this was the worst mo
 24. here we are two travelers from a distant futuristic world arrive on earth one is on a desperate mission to
 25. this movie sucked plain and simple okay so its basically about a girl that gets raped and to get revenge s
 26. some amusing humor some that falls flat some decent acting some that is quite atrocious this movie is simp
 27. the only words you need fear more than joe don baker if your thinking of watching a film are greydon clark
 28. this is simply put the worst movie i have ever seen it ranges from like hours and the box art was totally
 29. this inferior sequel based by the characters created by david selzer and harvey bernhardalso producer conc
 30. dreck about three beautiful women in california who go to cover some festival or something all the hotels
 31. noting the cast i recently watched this movie on tcm hoping for an underappreciated gem as i regard many f
 32. cheaplooking and ugly this film didnt even seem to entertain the kids in the audience except for one fairl
 33. a demented scientist girlfriend is decapitated so he brings her head back to life honest this is the plot
 34. this really was a waste of timethe movie has a weak plot the story is fragmented and ends very abruptly wi
 35. brief encounter is a ghastly and pointless remake of the david lean classic which was based on noel coward
 36. remember the good ol animated batman show from the s the one that people praised the one that people of al
 37. this movie is probably the worst i have seen bad acting bad script bad everything comparing it to mainly t
 38. somewhere on this site someone wrote that to get the best version of the works of jane austen one should s
 39. i know that in this episode theres other stuff apart from what i am going to discuss and in fact i think i
 40. so i know that i voted out of but really this deserves no more than half of a star i hated it it was so st
 41. i saw this film when it was first released the memory of how bad it was has stayed with me almost forty ye
 42. ill be short and to the point this movie was an insult to any one with a room temperature iq sorry liberal
 43. this movie causes more unintentional laughter than anything else ive ever seen really if you are a tolkie
 44. this film makes you really appreciate the invention of the fast forward button on your remote control its
 45. lets be honest shall we al gore no more truly cares about the environment than most folks care about conta
 46. liongate has yet to prove itself every single movie from lionsgate has been abysmal ive tried and tried to
 47. st watched out of dirbrad sykes mindless d movie about flesheating zombies in a story within a movie chror
 48. the world is facing imminent destruction and a suicide mission is sent to the sun to avert catastrophe by
 49. feeding the masses is just one of many recent mediocre zombie movies to be after your hardearned dollars s
 50. with the exception of about sublime minutes with hb warner on the celestial train this was minutes of jaw
 51. dear me where do i start the dad isnt anywhere near old enough to be the girls dad he corpses on camera ir
 52. before this made for tv movie began i had relatively low expectations thats because it was made after the
 53. an unoriginal overly predictable and only mildly entertaining low budget rehash of a scifi formula that we
 54. followup to s far better cleopatra jones has statuesque black actress tamara dobson returning to her signa
 55. if schanti had been a serious attempt at a film about the institution of slavery still prevalent in third

```
print("original texts:")
for i, text in enumerate(data):
    print(f"{i+1}.{text}")
```

```
original texts:  
1.sentiment  
2.review
```

```
nltk.download('punkt')  
word_tokenized_texts = [word_tokenize(text) for text in cleaned_texts]  
print("\nWord Tokenized Texts:")  
for i, tokens in enumerate(word_tokenized_texts):  
    print(f"{i+1}. {tokens}")  
  
sample_paragraph = cleaned_texts[0] # Using the first cleaned text as a sample  
sentences=sent_tokenize(sample_paragraph)  
print("\nSentence Tokenization:")  
for i, sentence in enumerate (sentences):  
    print(f"{i+1}.{sentence}")
```

```

9971. ['spoiler', 'this', 'movie', 'is', 'about', 'such', 'a', 'concept', 'williams', 'will', 'go', 'to', 'any
9972. ['perhaps', 'the', 'most', 'polished', 'and', 'accomplished', 'of', 'all', 'indian', 'films', 'pakeezah
9973. ['one', 'of', 'the', 'last', 'classics', 'of', 'the', 'french', 'new', 'wave', 'for', 'direction', 'cine
9974. ['in', 'a', 'penitentiary', 'four', 'prisoners', 'occupy', 'a', 'cell', 'carrã're', 'gã@rald', 'laroche
9975. ['just', 'as', 'tom', 'berenger', 'put', 'you', 'into', 'the', 'soul', 'of', 'sgt', 'barnes', 'he', 'has
9976. ['david', 'mamet', 'wrote', 'the', 'screenplay', 'and', 'made', 'his', 'directorial', 'debut', 'with',
9977. ['your', 'average', 'garden', 'variety', 'psychotic', 'nutcase', 'deliciously', 'essayed', 'with', 'unhi
9978. ['oftentimes', 'films', 'of', 'this', 'nature', 'come', 'across', 'as', 'a', 'mixed', 'bag', 'of', 'grea
9979. ['i', 'suggested', 'renting', 'this', 'movie', 'to', 'my', 'friend', 'and', 'he', 'obliged', 'since', 't
9980. ['for', 'me', 'this', 'movie', 'was', 'powerful', 'i', 'dont', 'want', 'to', 'be', 'a', 'spoiler', 'but
9981. ['the', 'film', 'of', 'artemisias', 'may', 'be', 'considered', 'treason', 'or', 'as', 'true', 'artistic',
9982. ['i', 'saw', 'this', 'movie', 'in', 'sixth', 'grade', 'around', 'christmas', 'time', 'and', 'i', 'was',
9983. ['almost', 'every', 'review', 'of', 'this', 'movie', 'id', 'seen', 'was', 'pretty', 'bad', 'its', 'not',
9984. ['i', 'first', 'read', 'pearl', 's', 'bucks', 'splendid', 'novel', 'in', 'my', 'ninth', 'grade', 'histor
9985. ['having', 'never', 'heard', 'of', 'this', 'film', 'until', 'i', 'saw', 'the', 'rental', 'dvd', 'i', 'as
9986. ['sweet', 'romantic', 'dramacomedy', 'about', 'stewart', 'and', 'sullivan', 'writing', 'love', 'letters
9987. ['i', 'thought', 'i', 'should', 'qualify', 'my', 'position', 'after', 'reading', 'other', 'reviews', 'th
9988. ['i', 'dont', 'think', 'this', 'movie', 'is', 'for', 'everyone', 'but', 'i', 'saw', 'it', 'this', 'week
9989. ['ive', 'seen', 'this', 'about', 'or', 'times', 'and', 'havent', 'regretted', 'it', 'homeward', 'bound',
9990. ['jared', 'diamond', 'made', 'a', 'point', 'in', 'the', 'first', 'episode', 'that', 'other', 'peoples',
9991. ['it', 'is', 'a', 'superb', 'swedish', 'film', 'it', 'was', 'the', 'first', 'swedish', 'film', 'ive', 's
9992. ['tripping', 'over', 'i', 'must', 'say', 'at', 'first', 'i', 'was', 'a', 'little', 'disappointed', 'in',
9993. ['military', 'training', 'films', 'are', 'becoming', 'so', 'common', 'that', 'they', 'are', 'becoming',
9994. ['the', 'movie', 'itself', 'made', 'me', 'want', 'to', 'go', 'and', 'call', 'someone', 'so', 'they', 'cc
9995. ['enjoyable', 'in', 'spite', 'of', 'leslie', 'howards', 'performance', 'mr', 'howard', 'plays', 'philip
9996. ['i', 'first', 'saw', 'this', 'movie', 'back', 'in', 'the', 'earlys', 'when', 'it', 'was', 'first', 'rel
9997. ['nyc', 'the', 'greenhouse', 'effect', 'vanished', 'oceans', 'grinding', 'unemployment', 'and', 'scarcit
9998. ['those', 'individuals', 'familiar', 'with', 'asian', 'cinema', 'as', 'a', 'whole', 'are', 'aware', 'tha
9999. ['the', 'kids', 'aged', 'to', 'got', 'such', 'a', 'huge', 'kick', 'out', 'of', 'this', 'film', 'that',
10000. ['i', 'so', 'love', 'this', 'movie', 'the', 'animation', 'is', 'great', 'for', 'a', 'pokã@mon', 'movie

```

Sentence Tokenization:

1.i had no background knowledge of this movie before i bought it but it sounded cool and ive been wanting to s

```

nltk.download('stopwords')
stop_words=set(stopwords.words('english'))
filtered_texts=[]
for tokens in word_tokenized_texts:
    filtered_tokens= [word for word in tokens if word not in stop_words]
    filtered_texts.append(filtered_tokens)
print("\nTests after Stopword Removal:")

```

```
for idx, tokens in enumerate(filtered_texts):  
    print(f"{idx+1}.{tokens}")
```

```

9983.['almost', 'every', 'review', 'movie', 'id', 'seen', 'pretty', 'bad', 'pretty', 'bad', 'actually', 'preti
9984.['first', 'read', 'pearl', 'bucks', 'splendid', 'novel', 'ninth', 'grade', 'history', 'class', 'enjoyed',
9985.['never', 'heard', 'film', 'saw', 'rental', 'dvd', 'bit', 'sceptical', 'many', 'films', 'past', 'good',
9986.['sweet', 'romantic', 'dramacomedy', 'stewart', 'sullavan', 'writing', 'love', 'letters', 'without', 'eit
9987.['thought', 'qualify', 'position', 'reading', 'reviews', 'movie', 'great', 'lot', 'great', 'elements', 'l
9988.['dont', 'think', 'movie', 'everyone', 'saw', 'weekend', 'seattle', 'thought', 'funny', 'havent', 'laught
9989.['ive', 'seen', 'times', 'havent', 'regretted', 'homeward', 'bound', 'typical', 'animal', 'movie', 'uniqu
9990.['jared', 'diamond', 'made', 'point', 'first', 'episode', 'peoples', 'world', 'didnt', 'animals', 'domest
9991.['superb', 'swedish', 'film', 'first', 'swedish', 'film', 'ive', 'seen', 'simple', 'deep', 'great', 'comb
9992.['tripping', 'must', 'say', 'first', 'little', 'disappointed', 'first', 'episodes', 'faith', 'show', 'abe
9993.['military', 'training', 'films', 'becoming', 'common', 'becoming', 'genre', 'unto', 'among', 'prominent
9994.['movie', 'made', 'want', 'go', 'call', 'someone', 'could', 'enjoy', 'extremely', 'funny', 'angelena', 'i
9995.['enjoyable', 'spite', 'leslie', 'howards', 'performance', 'mr', 'howard', 'plays', 'philip', 'flat', 'ur
9996.['first', 'saw', 'movie', 'back', 'earlys', 'first', 'released', 'room', 'view', 'also', 'newly', 'enchar
9997.['nyc', 'greenhouse', 'effect', 'vanished', 'oceans', 'grinding', 'unemployment', 'scarcity', 'water', 'p
9998.['individuals', 'familiar', 'asian', 'cinema', 'whole', 'aware', 'japan', 'renowned', 'notorious', 'hyper
9999.['kids', 'aged', 'got', 'huge', 'kick', 'film', 'gave', 'copy', 'kids', 'birthday', 'list', 'year', 'love
10000.['love', 'movie', 'animation', 'great', 'pokémon', 'movie', 'cgi', 'looks', 'awesome', 'love', 'music',

```

```

from nltk.stem import PorterStemmer, WordNetLemmatizer
from nltk.corpus import wordnet
from nltk.tag import pos_tag
import nltk
nltk.download('averaged_perceptron_tagger_eng')
nltk.download('wordnet')

```

```

stemmer=PorterStemmer()
lemmatizer=WordNetLemmatizer()

def get_wordnet_pos(treebank_tag):
    if treebank_tag.startswith('J'):
        return wordnet.ADJ
    elif treebank_tag.startswith('V'):
        return wordnet.VERB
    elif treebank_tag.startswith('N'):
        return wordnet.NOUN
    elif treebank_tag.startswith('R'):
        return wordnet.ADV
    else:

```

```
        return wordnet.NOUN
stemmed_texts=[]
for tokens in filtered_texts:
    stemmed_tokens=[stemmer.stem(token)for token in tokens]
    stemmed_texts.append(stemmed_tokens)

print("\nStemmed texts:")
for i,tokens in enumerate(stemmed_texts):
    print(f"{i+1}.{tokens}")

lemmatized_texts=[]
for tokens in filtered_texts:
    pos_tags=pos_tag(tokens)
    lemmantized_tokens=[]
    for word,tag in pos_tags:
        pos=get_wordnet_pos(tag)
        lemmantized_tokens.append(lemmatizer.lemmatize(word,pos))
    lemmatized_texts.append(lemmantized_tokens)

print("\nLemmatized Texts:")
for i,tokens in enumerate(lemmatized_texts):
    print(f"{i+1}.{tokens}")
```



```

9961.['fatih', 'akins', 'first', 'work', 'im', 'juli', 'fairly', 'good', 'create', 'really', 'gorgeous', 'ital
9962.['truly', 'remarkable', 'piece', 'cinematic', 'achievement', 'start', 'utterly', 'hook', 'true', 'story',
9963.['strange', 'feel', 'sit', 'alone', 'theater', 'occupy', 'parent', 'rollick', 'kid', 'felt', 'like', 'ins
9964.['back', 'root', 'like', 'heaven', 'real', 'value', 'life', 'swede', 'carve', 'message', 'appeal', 'every
9965.['interest', 'character', 'lot', 'tension', 'close', 'black', 'white', 'without', 'black', 'white', 'turr
9966.['another', 'spanish', 'movie', 'civil', 'war', 'time', 'tell', 'story', 'carol', 'lovely', 'play', 'dã@k
9967.['year', 'ernesto', 'che', 'guevara', 'cuban', 'citizen', 'last', 'five', 'yearsdisappears', 'face', 'ear
9968.['comedy', 'much', 'underlying', 'pain', 'sadness', 'succeed', 'others', 'fail', 'many', 'film', 'genre',
9969.['people', 'without', 'swift', 'mind', 'without', 'drop', 'balkan', 'blood', 'vein', 'dont', 'understand
9970.['watch', 'trailer', 'dvd', 'see', 'film', 'think', 'anyone', 'saw', 'watching', 'film', 'would', 'surpri
9971.['spoiler', 'movie', 'concept', 'williams', 'go', 'low', 'order', 'replay', 'football', 'game', 'haunt',
9972.['perhaps', 'polish', 'accomplished', 'indian', 'film', 'pakeezah', 'fall', 'trap', 'commonly', 'associat
9973.['one', 'last', 'classic', 'french', 'new', 'wave', 'direction', 'cineaste', 'jean', 'eustache', 'draw',
9974.['penitentiary', 'four', 'prisoner', 'occupy', 'cell', 'carrã're', 'gã@rald', 'laroche', 'use', 'company
9975.['tom', 'berenger', 'put', 'soul', 'sgt', 'barnes', 'do', 'thomas', 'beckett', 'think', 'world', 'fold',
9976.['david', 'mamet', 'write', 'screenplay', 'make', 'directorial', 'debut', 'house', 'game', 'character',
9977.['average', 'garden', 'variety', 'psychotic', 'nutcase', 'deliciously', 'essay', 'unhinged', 'glee', 'ste
9978.['oftentimes', 'film', 'nature', 'come', 'across', 'mixed', 'bag', 'great', 'work', 'along', 'slight', 'c
9979.['suggest', 'rent', 'movie', 'friend', 'oblige', 'since', 'already', 'see', 'film', 'say', 'okay', 'think
9980.['movie', 'powerful', 'dont', 'want', 'spoiler', 'friend', 'year', 'ago', 'like', 'brother', 'movie', 'br
9981.['film', 'artemisla', 'may', 'consider', 'treason', 'true', 'artistic', 'license', 'might', 'one', 'aver
9982.['saw', 'movie', 'sixth', 'grade', 'around', 'christmas', 'time', 'really', 'excite', 'see', 'heard', 'ge
9983.['almost', 'every', 'review', 'movie', 'id', 'see', 'pretty', 'bad', 'pretty', 'bad', 'actually', 'pretty
9984.['first', 'read', 'pearl', 'buck', 'splendid', 'novel', 'ninth', 'grade', 'history', 'class', 'enjoy', 'e
9985.['never', 'hear', 'film', 'saw', 'rental', 'dvd', 'bit', 'sceptical', 'many', 'film', 'past', 'good', 'er
9986.['sweet', 'romantic', 'dramacomedy', 'stewart', 'sullavan', 'write', 'love', 'letter', 'without', 'either
9987.['think', 'qualify', 'position', 'read', 'reviews', 'movie', 'great', 'lot', 'great', 'element', 'light',
9988.['dont', 'think', 'movie', 'everyone', 'saw', 'weekend', 'seattle', 'think', 'funny', 'havent', 'laugh',
9989.['ive', 'see', 'time', 'havent', 'regret', 'homeward', 'bind', 'typical', 'animal', 'movie', 'unique', 't
9990.['jar', 'diamond', 'make', 'point', 'first', 'episode', 'people', 'world', 'didnt', 'animal', 'domesticat
9991.['superb', 'swedish', 'film', 'first', 'swedish', 'film', 'ive', 'see', 'simple', 'deep', 'great', 'combi
9992.['trip', 'must', 'say', 'first', 'little', 'disappointed', 'first', 'episode', 'faith', 'show', 'abe', 't
9993.['military', 'training', 'film', 'become', 'common', 'become', 'genre', 'unto', 'among', 'prominent', 'of
9994.['movie', 'make', 'want', 'go', 'call', 'someone', 'could', 'enjoy', 'extremely', 'funny', 'angelena', 'c
9995.['enjoyable', 'spite', 'leslie', 'howard', 'performance', 'mr', 'howard', 'play', 'philip', 'flat', 'unir
9996.['first', 'saw', 'movie', 'back', 'earlys', 'first', 'release', 'room', 'view', 'also', 'newly', 'enchant
9997.['nyc', 'greenhouse', 'effect', 'vanish', 'ocean', 'grind', 'unemployment', 'scarcity', 'water', 'power',
9998.['individual', 'familiar', 'asian', 'cinema', 'whole', 'aware', 'japan', 'renowned', 'notorious', 'hyper
9999.['kid', 'age', 'get', 'huge', 'kick', 'film', 'give', 'copy', 'kid', 'birthday', 'list', 'year', 'love',
10000.['love', 'movie', 'animation', 'great', 'pokã@mon', 'movie', 'cgi', 'look', 'awesome', 'love', 'music',

```

```

from sklearn.feature_extraction.text import CountVectorizer, TfidfVectorizer

preprocessed_texts=[''.join(tokens) for tokens in lemmatized_texts]

count_vectorizer=CountVectorizer()
bow_matrix=count_vectorizer.fit_transform(preprocessed_texts)

print("\nBag of Words Matrix:")
print(bow_matrix.toarray())
print("Feature names:", count_vectorizer.get_feature_names_out())

tfidf_vectorizer=TfidfVectorizer()
tfidf_matrix = tfidf_vectorizer.fit_transform(preprocessed_texts)

print("\nTF-IDF Matrix:")
print(tfidf_matrix.toarray())
print("Feature names:", tfidf_vectorizer.get_feature_names_out())

```

Bag of Words Matrix:

```

[[0 0 0 ... 0 0 0]
 [0 0 0 ... 0 0 0]
 [0 0 0 ... 0 0 0]

```

...

```

[0 0 0 ... 0 0 0]
[0 0 0 ... 0 0 0]
[0 0 0 ... 0 0 0]

```

Feature names: ['abernalfirstamoresperrosfirstjuangerarddirectorposternamedianabrachoceciliasuareappearedianapl
'abhaydeolmeetattractivesohaalikhangreethellosistersettoneremarkabledebutfilmshivamnairsohamiddleclassgirlruna
'abhaydeolssecondfilmwriteimtiazalimaidendirectorialeffortshivamnairsohaprobablyfirstmeatyrolemeghagirlrunaway
...
'zuwarriordefinitelyshouldveanimatedseriesmovielikewatcholdanimeacidthemoviestartnowherepeopleflyaroundfightme
'âdisappointinglydoesntsomeoneelsegoodproductionvaluethaihorrorflickblandscriptnevergeneratesuspendeddirectorap
'âžiâžekmakemoviefunwatchunanswerablequestiononehelpaskoutrageousâžiâžeksscreenpresencedocumentary

TF-IDF Matrix:

```

[[0. 0. 0. ... 0. 0. 0.]
 [0. 0. 0. ... 0. 0. 0.]
 [0. 0. 0. ... 0. 0. 0.]

```

...

```

[0. 0. 0. ... 0. 0. 0.]

```

```
[0. 0. 0. ... 0. 0. 0.]
[0. 0. 0. ... 0. 0. 0.]]
Feature names: ['abernalfirstamoresperrosfirstjuangerarddirectorposternamedianabrachoceciliasuaresappeardianapl
'abhaydeolmeetattractivesohaalikhangreethellosistersettoneremarkabledebutfilmshivamnairsohamiddleclassgirlruna
'abhaydeolssecondfilmwriteimtiazaalimaidendirectorialeffortshivamnairsohaprobablyfirstmeatyrolemeghagirrunaway
...
'zuwarriordefinitelyshouldveanimatedseriesmovielikewatcholdanimeacidthemoviestartnowherepeopleflyaroundfightme
'adisappointinglydoesntsomeneelsegoodproductionvaluethaihorrorflickblandscriptnevergeneratesuspensedirectorap
'âziâžekmakemoviefunwatchunanswerablequestiononehelpaskoutrageousâziâžekssviewâziâžekssscreenpresencedocumentary
```

```
from sklearn.model_selection import train_test_split

from sklearn.linear_model import LogisticRegression

from sklearn.naive_bayes import MultinomialNB

from sklearn.svm import SVC

from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

data=pd.read_csv("User/sDELL/Downloads/Review.csv/Review.csv")
pd.DataFrame(data)
print("data for classifcation")
```

data for classifcation

```
from sklearn.preprocessing import LabelEncoder

# Define X and y using preprocessed_texts and encode sentiment labels
X = preprocessed_texts
le = LabelEncoder()
y = le.fit_transform(data['sentiment'])

# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=

# Initialize and fit TfidfVectorizer (corrected capitalization)
vectorizer = TfidfVectorizer()
```

```

X_train_vec = vectorizer.fit_transform(X_train)
X_test_vec = vectorizer.transform(X_test)

# Corrected f-string formatting
print(f"Training set size: {X_train_vec.shape}")
print(f"Test set size: {X_test_vec.shape}")

classifiers = {
    'Logistic Regression': LogisticRegression(max_iter=200),
    'Multinomial Naive Bayes': MultinomialNB(),
    'Support Vector Machine': SVC(kernel='linear')
}

results = {}
for name, clf in classifiers.items():
    clf.fit(X_train_vec, y_train)
    y_pred = clf.predict(X_test_vec) # Corrected assignment
    accuracy = accuracy_score(y_test, y_pred) # Corrected assignment
    results[name] = accuracy
    print(f"\n{name} Accuracy: {accuracy:.2f}")
    print(classification_report(y_test, y_pred))

# Ensure matplotlib is imported for plotting
import matplotlib.pyplot as plt

```

Training set size: (7000, 8433)

Test set size: (3000, 8433)

Logistic Regression Accuracy: 0.52

	precision	recall	f1-score	support
0	0.52	1.00	0.68	1558
1	1.00	0.00	0.01	1442
accuracy			0.52	3000
macro avg	0.76	0.50	0.35	3000
weighted avg	0.75	0.52	0.36	3000

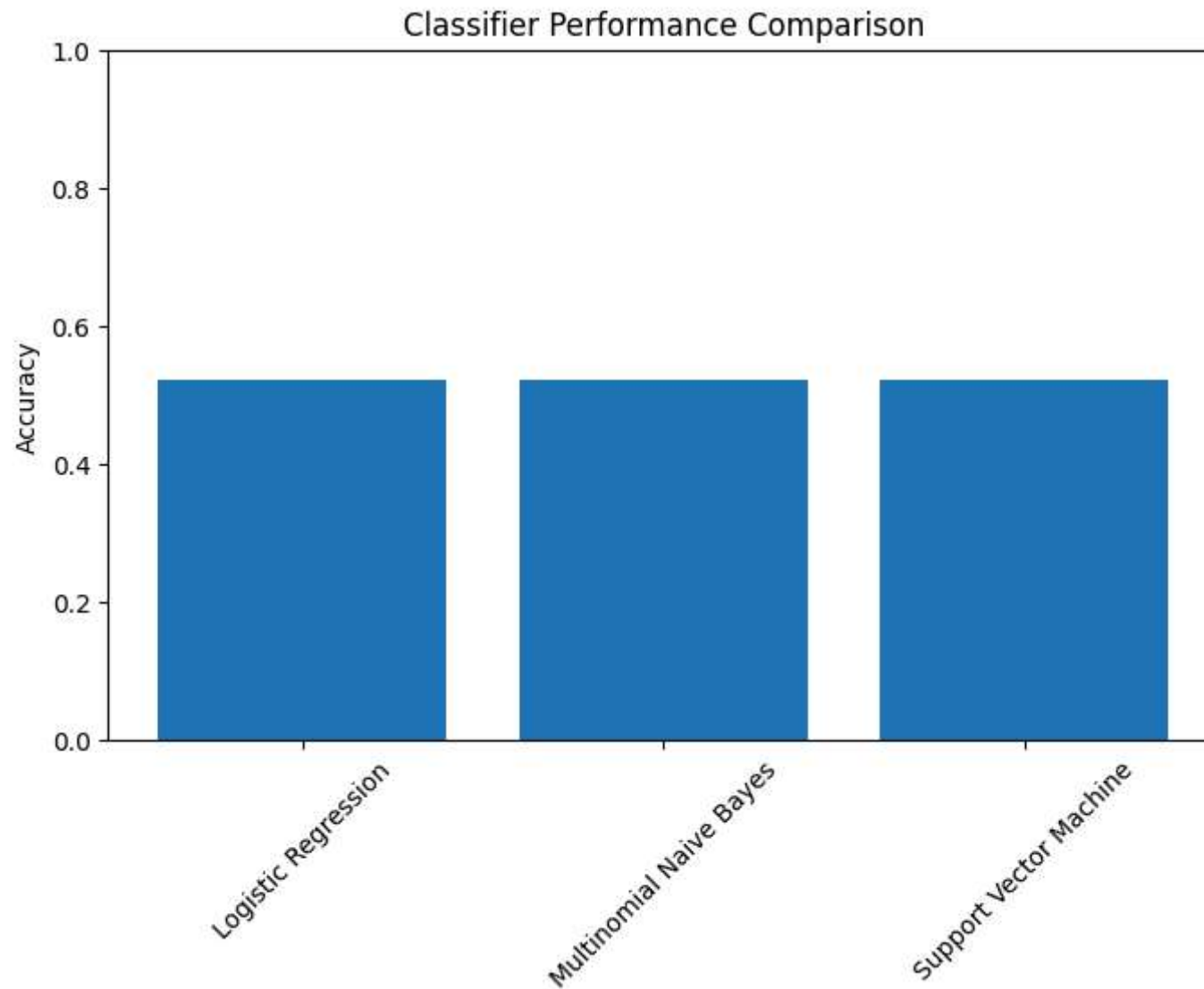
Multinomial Naive Bayes Accuracy: 0.52

	precision	recall	f1-score	support
0	0.52	1.00	0.68	1558
1	1.00	0.00	0.01	1442
accuracy			0.52	3000
macro avg	0.76	0.50	0.35	3000
weighted avg	0.75	0.52	0.36	3000

Support Vector Machine Accuracy: 0.52

	precision	recall	f1-score	support
0	0.52	1.00	0.68	1558
1	1.00	0.00	0.01	1442
accuracy			0.52	3000
macro avg	0.76	0.50	0.35	3000
weighted avg	0.75	0.52	0.36	3000

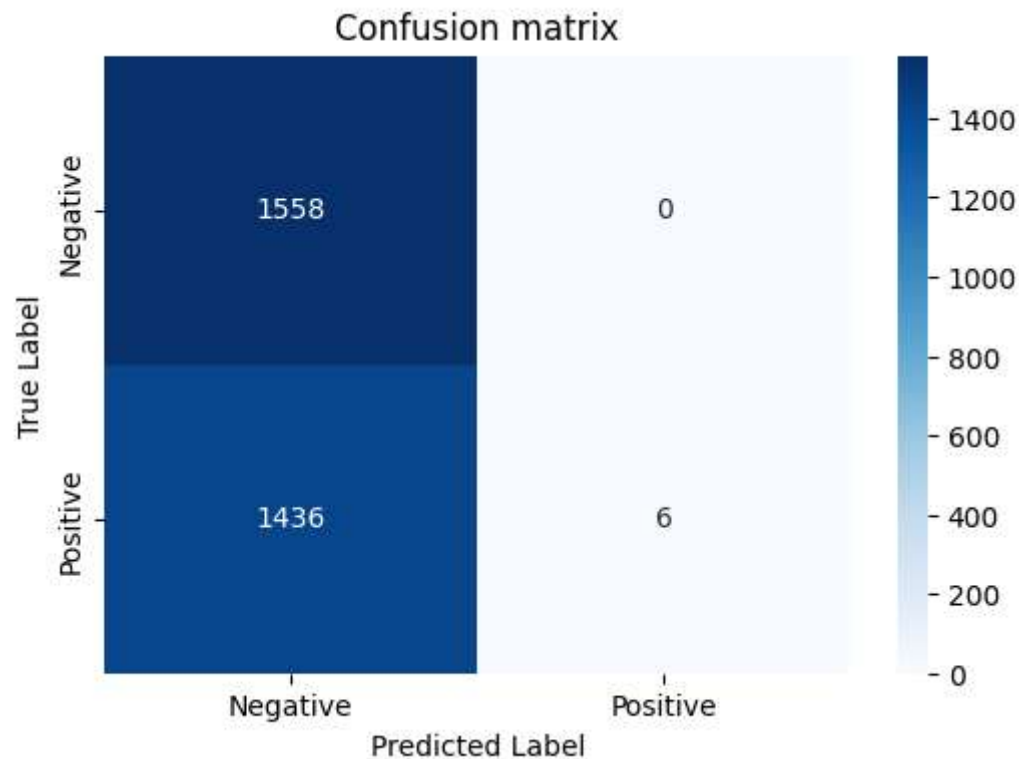
```
plt.figure(figsize=(8, 5))
plt.bar(results.keys(), results.values())
plt.title('Classifier Performance Comparison')
plt.ylabel('Accuracy')
plt.ylim(0, 1)
plt.xticks(rotation=45)
plt.show()
```



```
best_model=LogisticRegression(max_iter=200)
best_model.fit(X_train_vec, y_train)
y_pred=best_model.predict(X_test_vec)

cm=confusion_matrix(y_test, y_pred)
plt.figure(figsize=(6, 4))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
```

```
xticklabels=['Negative', 'Positive'],  
yticklabels=['Negative', 'Positive'])  
plt.title('Confusion matrix')  
plt.ylabel('True Label')  
plt.xlabel('Predicted Label')  
plt.show()
```



```
%pip install gensim
```

Collecting gensim

Downloading gensim-4.4.0-cp312-cp312-manylinux_2_24_x86_64.manylinux_2_28_x86_64.whl.metadata (8.4 kB)
Requirement already satisfied: numpy>=1.18.5 in /usr/local/lib/python3.12/dist-packages (from gensim) (2.0.2)
Requirement already satisfied: scipy>=1.7.0 in /usr/local/lib/python3.12/dist-packages (from gensim) (1.16.3)
Requirement already satisfied: smart_open>=1.8.1 in /usr/local/lib/python3.12/dist-packages (from gensim) (7.6.0)
Requirement already satisfied: wrapt in /usr/local/lib/python3.12/dist-packages (from smart_open>=1.8.1->gensim) (1.16.0)
Downloading gensim-4.4.0-cp312-cp312-manylinux_2_24_x86_64.manylinux_2_28_x86_64.whl (27.9 MB)
27.9/27.9 MB 54.6 MB/s eta 0:00:00

Installing collected packages: gensim
 Successfully installed gensim-4.4.0

```

from gensim.models import Word2Vec # Corrected import statement
sentences = lemmatized_texts # Use pre-processed, tokenized text
model = Word2Vec(sentences, vector_size=50, window=3, min_count=1, sg=1)

# Assuming 'nlp' or a similar word exists in the vocabulary for demonstration
# If 'nlp' is not in the vocabulary, you might get a KeyError.
# We can use a common word or check if a word exists before accessing.
# For now, let's try a word from the dataset, e.g., 'movie' if it's frequent.
# If 'movie' is not a good example, we can adjust after first run.

# Example: try to find 'movie' or handle KeyErrors gracefully
if 'movie' in model.wv:
    vector_movie = model.wv['movie']
    print("Vector for 'movie' (first 10 dimensions):\n", vector_movie[:10])
    similar_to_movie = model.wv.most_similar('movie', topn=3)
    print("\nWords most similar to 'movie':", similar_to_movie)
else:
    print("'movie' not found in vocabulary. Please choose a word present in the text.")

# If you still want to try 'nlp' as a general term and expect it in corpus:
# if 'nlp' in model.wv:
#     vector_nlp = model.wv['nlp']
#     print("Vector for 'nlp' (first 10 dimensions):\n", vector_nlp[:10])
#     similar_to_nlp = model.wv.most_similar('nlp', topn=3)
#     print("\nWords most similar to 'nlp':", similar_to_nlp)
# else:
#     print("'nlp' not found in vocabulary.")

Vector for 'movie' (first 10 dimensions):
[-0.16539253 -0.2434332  0.13987853  0.5906573  -0.0853436  -0.72270566
 0.21359931  0.9415223  -0.4480142  -0.21903715]

Words most similar to 'movie': [('film', 0.9501811861991882), ('anime', 0.8980541229248047), ('films', 0.892679

```